

Saturday
7-02-2026

- Global context refers to the default environment where code runs.
- When the Javascript engine starts executing code.

MTWTFSS

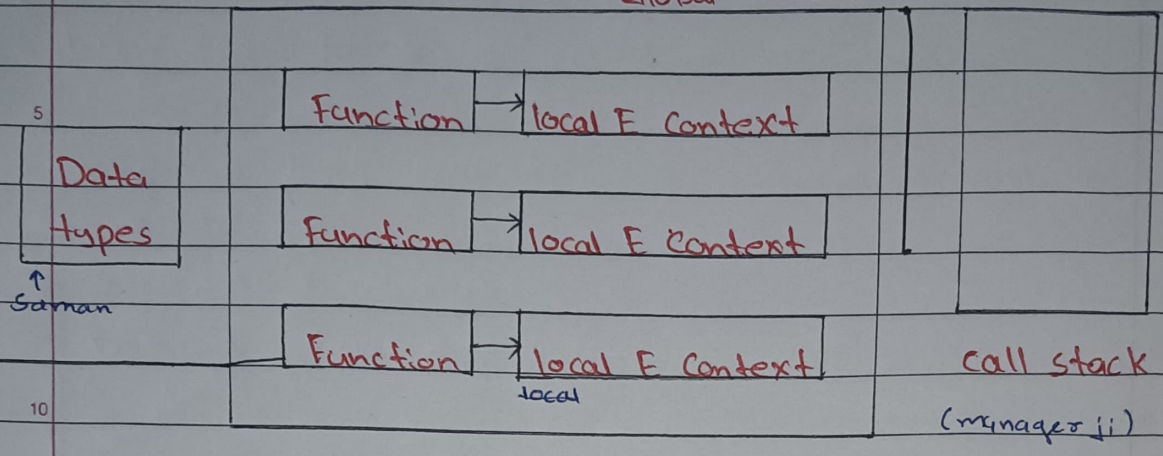
Date : / /
Page No: _____

Javascript Classes

QA :- Quality Analysis

(factory me jo kam hota hai)

Global context



- local execution context, more formally known as a function Execution Context (FEC), is a private environment created by the Javascript engine every time a function is called.

- Every function has there own.

actual work	Board
main function	variable, function
run	Ko rakhana
execution thread	memory

Const number = 5;

Function addTwo(num) {

return num + 2

}

memory
number=5
addTwo(num)

Const valueOne = addTwo(number)

new variable who calls the function

mining
factory

main thread:- Check the code from top to bottom
like interpreter

MTWTFSS

Date : / /

Page No: _____

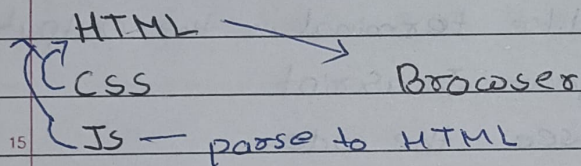
How all work together

- When new variable calls the function.
- addTwo number executes to function locally.
- But the first it is add to the callstack
- then memory provide Data to the function which is sitting in the callstack.
- and the memory is the global execution memory.
- then execution thread execute line by line.

Programming is all about

- data - - - string, function, boolean
- Processing - - loop, function, variable

Browser only understand



JS Engine Names

- V8 engine → chrome
- Spider Monkey → Firefox
- Safari → Webkit

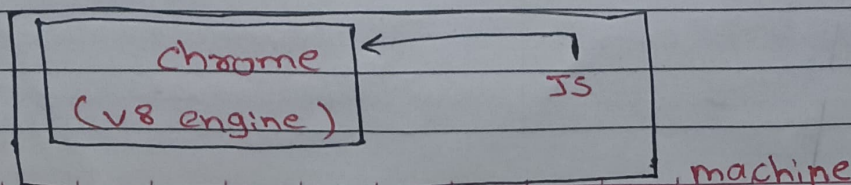
without engine you can't run JS in your browser.

- Which compiler need to run code

C, C++ = CPP Compiler

Java = Java Compiler

Javascript = JS compiler Engine.



nodeJs inventor

Ryan Dahl

- It's not a framework or not a library
- It's run time environment for js
- for run js ~~is~~ runs outside the browser

- there is two way to add JavaScript in your HTML file

- external JavaScript
- Internal JavaScript

Developer tools

- Element $\Rightarrow$ for HTML CSS
- Console $\Rightarrow$ it's like terminal or the output of your JavaScript
- Sources $\Rightarrow$ to see the code
Kaha kaha se code hora vo
- network $\Rightarrow$ to see if your browser calles external api's
- performance $\Rightarrow$ as name says

- Js is loose typed language.
- TypeScript is strongly typed language.
- and to browsers don't understand TypeScript you have to convert TypeScript in to JavaScript then it's works.

typescript is just pro version of JavaScript

Functions - set of instruction

- single source + use again and again.

function addNumbers() {

name of function

function addNumbers (num1 , num2) {

var result = num1 * num2;

Parameters

console.log ('Result is' , result);

}

addNumbers (2,3);

Conditionals

- Kisi perticular conditions ke upper kisi perticular block of code ko run karna.

(True & false are the Boolean value)

Fancy word :- Verbose

using more words than necessary to express

Ex,

Var age = 22;

Var childCondition = age <= 12;

Var teenCondition = age <= 19;

Var adultCondition = age <= 40;

Var seniorCondition = age > 40;

if (childCondition) {

console.log ('You are a child');

}

if (teenCondition) {

console.log ('You are a teen');

}

```

if (teenCondition)
if (adultCondition) {
  Console.log ('You are an adult');
}
if (seniorCondition) {
  Console.log ('You are a senior');
}

```

Output

you are an -adult

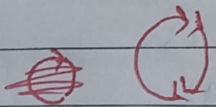
&& check both the condition

|| check either this or this
dono mese kon sa bhi chalega

! to represent not

else if use if you have two condition
output But you want to print
base on condition.
if condition matches print this or this.

loops



gol-gol

- doing thing again and again

- | | |
|-----------------|--------------|
| - for loop | - ForIn loop |
| - While loop | - Iterators |
| - Do while loop | - Filters |
| - ForEach loop | - Reduce |
| - ForOf loop | - Entries |
| - map | - Keys |

for loop

Syntax

```
for (initializer ; condition ; increment) {
    // code jisko loop karwana hai
}
```

example,

```

      init      Condition      increment
for ( var x = 1 ; x <= 10 ; x = x + 1 ) {
    console.log ('Value of x', x);
}
```

- Condition are always be boolean
- increment mean add +1 in next iteration until condition matches.

Dry Run

x	condition = $x \leq 10$	$x = x + 1$	Console
1	$1 \leq 10$ true		1
2	$2 \leq 10$ true		2
3	$3 \leq 10$ true		3
4	$4 \leq 10$ true		4
⋮	⋮		⋮
10	$10 \leq 10$ true		10
11	$11 \leq 10$ false		

Loop tut gai

- loop check the condition ~~to to~~ 11th time
- But run the code only 10th time

Use:- When you exactly know Kithi-baar code ko ghumana hai, matlab run karana hai.

use:- code ko tab tak ghumaao jab tak kaam
na pura hojaye

- only know the condition but don't
know how many iterations

SHREE

Date: / / Page No. _____

While loop

ex,

When you downloading a file, you don't know
how much time it's gonna take, but end ~~get~~ goal
is you get the file from the server.

get random number

```
function getRandomNumber (min, max) {  
    return Math.floor (Math.random () *  
                        (max - min + 1)) + min;  
}
```

example,

```
var fileSize = 1024;
```

```
var currentFileDownloaded = 0;
```

```
while (currentFileDownloaded < fileSize) {  
    console.log ('File ko Download krte Jao');  
    currentFileDownloaded = currentFileDownloaded  
                            + 512;  
}
```

Or using random number

```
while (currentFileDownloaded < fileSize) {  
    console.log ('File ko Download krte Jao');  
    currentFileDownloaded = currentFileDownloaded +  
                            getRandomNumber (20, 40);  
}
```

- check the condition then iterate

use:- Do-while ek toh ki while loop hi hai, but

First he execute the code the check the condition

- It ensure that you code at least one time runs.

SHREE

Date: / / Page No. _____

do-while loop

Example,

```
Var fileSize = 1024;
```

```
Var currentfileDownloaded = 0;
```

```
do {
```

```
  console.log('file ko Download krde jao');
```

```
  currentfileDownloaded = currentfileDownloaded + 512;
```

```
} while
```

```
(currentfileDownloaded < fileSize);
```

example

```
console.log('Age is', age);
```

```
var age = 32;
```

```
console.log('Age is', age);
```

```
console.log('Age is', age);
```

```
var age = 32;
```

same
line ↓
by line

age (variable)	undefined
memory phase	code phase

Output:- Age is undefined.

```
var age = 32;
```

```
console.log('Age is', age);
```

age	32
memory phase	code phase

Output, Age is 32

pehle memory
phase pehle
run krta hai
ise Hoisting
Khte hai

Hoisting is a javascript mechanism where variable, function, and class declarations are conceptually moved to the top of their containing scope (Script or function) during the compilation phase, before code execution.

- This allows functions to be used before they are defined and allows variable to be accessed without throwing error, though var variable return undefined.

Example,

```
age = 45;
```

```
console.log('Age is', age);
```

```
var age = 32;
```

```
hello();
```

```
console.log('Age is', age);
```

```
function hello() {
```

```
  console.log('This is hello');
```

```
}
```

• Output ↴

Age is 45

This is hello

Age is 32

example

```
console.log ('value of age is', age);
```

```
var age = 45;
```

```
console.log ('Adding 5 to 10', addFive(10));
```

```
console.log ('value of age is', age);
```

call the
function

```
function addfive(number) {
```

```
  var result = number + 5;
```

```
  return result;
```

```
}
```

Dry Run

memory phase	code phase										
<u>age</u> = <u>undefined</u> 45	undefined										
<pre>function <u>addfive</u>(number){ var <u>result</u> = number + 5; return result; }</pre>	<table border="1"> <thead> <tr> <th>MP</th><th>CP</th></tr> </thead> <tbody> <tr> <td>number = 10</td><td></td></tr> <tr> <td>undefined</td><td></td></tr> <tr> <td>result = 15</td><td></td></tr> <tr> <td>undefined</td><td></td></tr> </tbody> </table> addFive ka execution context find 45	MP	CP	number = 10		undefined		result = 15		undefined	
MP	CP										
number = 10											
undefined											
result = 15											
undefined											

Output:-

value of age is undefined

Adding 5 to 10 15

value of age is 45